



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251214
Nama Lengkap	Raffiel Gideon Nyolo Nyolo
Minggu ke / Materi	12/ Tipe Data Dictionary

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

A. Pendahuluan Dictionary

Dictionary mirip dengan list, tetapi lebih umum. Sementara indeks dalam dictionary dapat berupa apapun, indeks dalam list harus berupa integer. Dictionary terdiri dari pasangan kunci:nilai. Kunci harus berbeda, tidak boleh ada dua kunci yang sama dalam dictionary.

Dictionary juga dapat dianggap sebagai pemetaan antara kumpulan nilai dan kumpulan indeks, masing-masing memetakan nilai. Pasangan nilai kunci, atau "item", adalah istilah yang digunakan untuk menggambarkan asosiasi kunci dan nilai tersebut.

Sebagai contoh, kita akan membuat kamus yang menggabungkan kata-kata dalam bahasa Inggris dan Spanyol. Dengan demikian, data string akan berfungsi sebagai kunci dan nilai. Sepertinya setiap kata yang digunakan dalam bahasa Inggris memiliki arti yang sama dalam bahasa Spanyol.

Karena dict adalah fitur python yang sudah ada, harus dihindari untuk menggunakannya sebagai nama variabel. Tujuannya adalah untuk membuat dictionary baru yang kosong.

```
>>> eng2sp = dict()
>>> print (eng2sp)
Output: {}
```

Tanda kurung kurawal { } digunakan untuk merepresentasikan dictionary kosong. Untuk menambahkan item dalam dictionary, dapat menggunakan kurung kotak [].

```
>>> eng2sp['one'] = 'uno'
```

Dapat dilihat pada baris diatas membuat item yang memetakan dari kunci **“satu”** ke nilai **“uno”**. Jika mencetak *dictionary* tadi, maka akan terlihat pasangan nilai kunci antara kunci dan nilai.

Format output yang digunakan juga merupakan format input. Misalkan kita membuat dictionary baru dengan tiga item dan melakukan pencetakan hasil yang didapatkan berupa keseluruhan data dari dictionary tersebut.

```
>>> eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}
>>> print(eng2sp)
Output: {'one': 'uno', 'three': 'tres', 'two': 'dos'}
```

Secara umum urutan item pada dictionary tidak dapat diprediksi, tetapi tidak menjadi masalah utama karena elemen dalam dictionary tidak pernah diberikan indeks dengan indeks integer. Sebagai gantinya dapat menggunakan kunci untuk mencari nilai yang sesuai (*corresponding values*).

Berikut ini merupakan contoh urutan yang tidak terlalu berpengaruh dalam dictionary (contoh 1), jika terdapat kunci yang tidak ada dalam dictionary (contoh 2), fungsi `len` pada dictionary (contoh 3):

Contoh 1:

```
>>> print(eng2sp['two'])
'dos'
```

Contoh 2:

```
>>> print(eng2sp['four'])
KeyError: 'four'
```

Contoh 3:

```
>>> len(eng2sp)
3
```

Operator `in` dalam dictionary mengembalikan nilai benar (`true`) atau salah (`false`) sesuai dengan kunci yang ada dalam dictionary.

```
>>> 'one' in eng2sp
True
>>> 'uno' in eng2sp
False
```

Untuk mengetahui nilai yang ada pada dictionary dapat menggunakan salah satu metode `values`. Metode ini akan mengembalikan nilai sesuai dengan tipe datanya dan dikonversikan dalam list dan digunakan dalam operator `in`.

Operator **in** menggunakan algoritma yang berbeda untuk *list* dan *dictionary*. Untuk *list* menggunakan algoritma pencarian linear. Saat *list* bertambah, waktu pencarian juga akan bertambah. Dalam *dictionary*, Python menggunakan algoritma yang disebut *Hash Table*, operator **in** membutuhkan waktu yang sama untuk memprosesnya dan tidak memperdulikan item yang ada didalam *dictionary*.

B. Dictionary sebagai set penghitung (counters)

Sebagai contoh, misalkan kita diberikan sebuah string dan diminta untuk menghitung jumlah kemunculan setiap huruf di dalamnya. Ada beberapa cara yang bisa digunakan untuk menyelesaikan masalah tersebut, di antaranya membuat 26 variabel berbeda untuk setiap huruf alfabet, menggunakan sebuah list yang berisi 26 elemen dan menggunakan dictionary dengan huruf sebagai key dan jumlah kemunculannya sebagai value.

Walaupun ketiga cara tersebut menghasilkan perhitungan yang sama, masing-masing memiliki metode dan proses pengerjaan yang berbeda.

Implementasi dilakukan dengan menggunakan perhitungan (*computation*). Penggunaan komputasi model dictionary lebih praktis, dimana kita tidak perlu mengetahui huruf mana yang akan muncul dalam string. Hanya perlu memberikan ruang untuk huruf yang akan muncul. Berikut ini contohnya:

```
word = 'brokolikukus'
d = dict()
for c in word:
    if c not in d:
        d[c] = 1
    else:
        d[c] = d[c] + 1
print(d)
```

Output: {'b': 1, 'r': 1, 'o': 2, 'k': 3, 'l': 1, 'i': 1, 'u': 2, 's': 1}

Dictionary memiliki metode yang disebut `get` yang mengambil kunci dan nilai default. Jika kunci muncul di dictionary, akan mengembalikan nilai yang sesuai; jika tidak maka akan mengembalikan nilai default. Contoh:

```
>>> counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
>>> print(counts.get('jan', 0))
100
>>> print(counts.get('tim', 0))
0
```

Dengan menggunakan *get* kita dapat menulis *loop histogram* secara lebih ringkas.

```
kata = 'brokolikukus'
d = dict()
for c in kata:
    d[c] = d.get(c,0) + 1
print(d)
```

C. Dictionary dan File

Dictionary dapat digunakan untuk menghitung berapa banyak kata yang muncul dalam file yang terdiri dari beberapa teks tertulis. Mari kita mulai dengan file kata yang sangat sederhana yang diambil dari naskah Romeo dan Juliet.

Pertama, teks yang disingkat dan disederhanakan tanpa tanda baca akan digunakan, dan kemudian akan digunakan teks adegan dengan tanda baca.

```
But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief
```

Python yang dapat membaca baris-baris dalam file, memecah setiap baris menjadi daftar kata, lalu melakukan perjalanan melalui setiap baris dan menggunakan dictionary untuk menghitung setiap kata.

Misalnya kita menggunakan dua perulangan *for*. Perulangan luar digunakan untuk membaca setiap baris pada file, sedangkan perulangan dalam digunakan untuk memeriksa setiap kata di dalam baris tersebut.

Pola seperti ini disebut *nested loop* atau perulangan bersarang, karena terdapat perulangan di dalam perulangan lainnya. Perulangan dalam berjalan lebih cepat karena dieksekusi berulang kali pada setiap iterasi perulangan luar.

Dengan kombinasi kedua perulangan tersebut, program dapat memproses seluruh kata pada setiap baris file secara menyeluruh.

```
fname = input('Enter the file name: ')
try:
    fhand = open(fname)
except:
    print('File cannot be opened:', fname)
    exit()

counts = dict()
for line in fhand:
    words = line.split()
    for word in words:
        if word not in counts:
            counts[word] = 1
        else:
            counts[word] += 1
print(counts)
```

python count1.py
Enter the file name: romeo.txt
{'and': 3, 'envious': 1, 'already': 1, 'fair': 1, 'is': 3, 'through': 1, 'pale': 1, 'yonder': 1, 'what': 1, 'sun': 2, 'Who': 1, 'But': 1, 'moon': 1, 'window': 1, 'sick': 1, 'east': 1, 'breaks': 1, 'grief': 1, 'with': 1, 'light': 1, 'It': 1, 'Arise': 1, 'kill': 1, 'the': 3, 'soft': 1, 'Juliet': 1}

Ketika program diatas dijalankan, output yang akan didapatkan data dump jumlah semua dalam urutan hash yang tidak disortir. Pada statement else digunakan model penulisan yang lebih singkat untuk menambahkan variable. `counts[word] += 1` sama dengan `counts[word] = counts[word] + 1`. Metode lain dapat digunakan untuk mengubah nilai suatu variabel dengan jumlah yang diinginkan, misalnya dengan menggunakan `-`, `*`, dan `/`.

D. Looping dan Dictionary

Dalam statement *for*, *dictionary* akan bekerja dengan cara menelusuri kunci yang ada didalamnya. *Looping* ini akan melakukan pencetakan setiap kunci sesuai dengan hubungan nilainya.

```
counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
for key in counts:
    print(key, counts[key])
```

output:
jan 100
chuck 1
annie 42

Sebagai hasilnya, kunci tidak berada dalam pola urutan tertentu. Beberapa idiom looping yang disebutkan sebelumnya dapat digunakan dengan pola ini. Sebagai contoh, jika kita ingin menemukan semua entri di dictionary dengan nilai di atas 10, kita harus menggunakan kode program berikut:

```
counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
for key in counts:
    if counts[key] > 10 :
        print(key, counts[key])
```

Output:
jan 100
annie 42

Pada looping for yang melalui kunci dictionary, perlu ditambahkan operator indeks untuk mengambil nilai yang sesuai untuk setiap kunci.

Untuk menampilkan key dictionary secara alfabet, key terlebih dahulu diambil menggunakan keys() lalu disimpan ke dalam list dan diurutkan dengan sort(). Setelah itu, program melakukan perulangan untuk menampilkan setiap pasangan key dan value sesuai urutan alfabet. Implementasi dalam kode dapat dilihat dibawah ini:

```
counts = { 'chuck' : 1 , 'annie' : 42, 'jan': 100}
lst = list(counts.keys())
print(lst)
lst.sort()
for key in lst:
    print(key, counts[key])
```

Output:
['jan', 'chuck', 'annie']
annie 42
chuck 1
jan 100

E. Looping dan Dictionary

Pada bagian sebelumnya, contoh file yang digunakan sudah tidak memiliki tanda baca. Namun, pada bagian ini kita akan menggunakan file yang masih memiliki tanda baca lengkap, yaitu file romeo.txt.

```
But, soft! what light through yonder window breaks?
It is the east, and Juliet is the sun.
Arise, fair sun, and kill the envious moon,
Who is already sick and pale with grief,
```

Python memiliki fungsi split yang digunakan untuk memisahkan kata berdasarkan spasi sehingga setiap kata menjadi token terpisah. Karena itu, kata seperti "soft!" dan "soft" akan dianggap berbeda dan disimpan sebagai entri terpisah pada dictionary. Hal yang sama juga berlaku pada huruf kapital. Contohnya, "who" dan "Who" dianggap sebagai dua kata berbeda sehingga dihitung secara terpisah. Untuk mengatasi hal tersebut, dapat digunakan method string lain seperti lower, punctuation, dan translate. Di antara method tersebut, translate() termasuk yang paling jarang diperhatikan, tetapi cukup berguna dalam pengolahan teks.

```
line.translate(str.maketrans(fromstr, tostr, deletestr))
```

Untuk membuat string kosong untuk fromstr dan tostr, dan untuk deletetr, hapus semua karakter yang ada dalam deletetr. Meskipun kita tidak akan menentukan tostr, kita akan menggunakan parameter deletetr untuk menghapus semua tanda baca. Python akan mengetahui bagian mana yang dianggap sebagai "tanda baca" secara otomatis.

```
import string
fname = input('Enter the file name: ')
try:
    fhand = open(fname)
except:
    print('File cannot be opened:', fname)
    exit()

counts = dict()
for line in fhand:
    line = line.rstrip()
    line = line.translate(line.maketrans('', '', string.punctuation))
    line = line.lower()
    words = line.split()
    for word in words:
        if word not in counts:
            counts[word] = 1
        else:
            counts[word] += 1
print(counts)
```

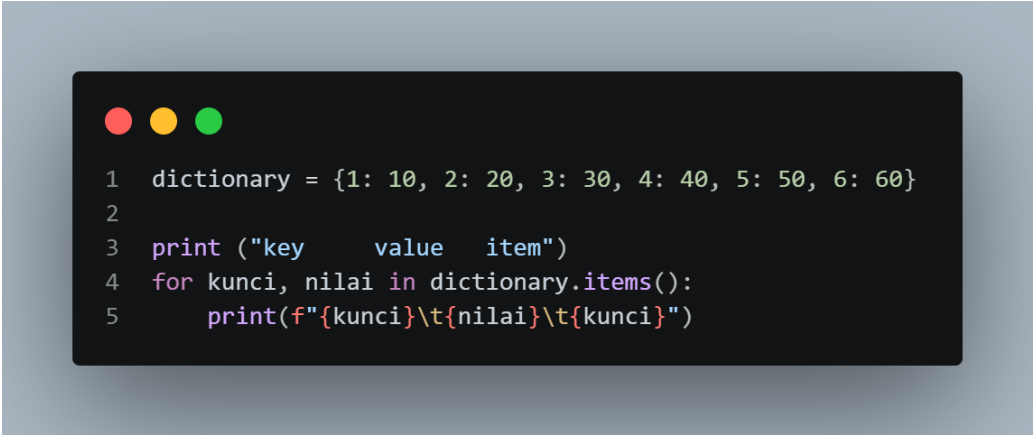
Enter the file name: romeo-full.txt
{'swearst': 1, 'all': 6, 'afeard': 1, 'leave': 2, 'these': 2, 'kinsmen': 2, 'what': 11, 'thinkst': 1, 'love': 24, 'cloak': 1, 'a': 24, 'orchard': 2, 'light': 5, 'lovers': 2, 'romeo': 40, 'maiden': 1, 'whiteupturned': 1, 'juliet': 32, 'gentleman': 1, 'it': 22, 'leans': 1, 'canst': 1, 'having': 1, ...}

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github: <https://github.com/fieliel-star/tugas-prakAlpro-minggu1-71251214.git>

SOAL 1



```
1 dictionary = {1: 10, 2: 20, 3: 30, 4: 40, 5: 50, 6: 60}
2
3 print ("key    value    item")
4 for kunci, nilai in dictionary.items():
5     print(f"{kunci}\t{nilai}\t{kunci}")
```

Penjelasan:

Pada baris pertama, dibuat sebuah dictionary bernama dictionary yang berisi pasangan key dan value, misalnya 1:10, 2:20, hingga 6:60. Setelah itu, program mencetak judul kolom yaitu "key value item" agar output lebih mudah dibaca dan sesuai dengan contoh soal yang diberikan.

Selanjutnya, program menggunakan perulangan for dengan method items() untuk mengambil setiap pasangan key dan value dari dictionary secara bersamaan. Variabel kunci menyimpan key, sedangkan nilai menyimpan value. Pada bagian print, digunakan f-string dan karakter \t (tab) untuk memberikan jarak antar kolom sehingga hasil output terlihat lebih rapi. Nilai yang ditampilkan terdiri dari key, value, dan key kembali pada kolom terakhir.

Output:

```
PS D:\Semester_2\Praktikum
key    value  item
1      10    1
2      20    2
3      30    3
4      40    4
5      50    5
6      60    6
```

SOAL 2

```
1 lista = ['red', 'green', 'blue']
2 listb = ['#FF0000', '#008000', '#0000FF']
3
4 hasil = {}
5 for lista, listb in zip(lista, listb):
6     hasil[lista] = listb
7
8 print(hasil)
```

Penjelasan:

Pada awal program, terdapat lista yang berisi nama warna seperti "red", "green", dan "blue", serta listb yang berisi kode warna dalam format hexadecimal. Setelah itu, dibuat dictionary kosong bernama hasil untuk menyimpan pasangan data dari kedua list tersebut.

Pada bagian `for lista, listb in zip(lista, listb):`, fungsi `zip()` digunakan untuk menggabungkan dua list berdasarkan urutan indeksinya. Artinya, elemen pertama dari lista akan dipasangkan dengan elemen pertama dari listb, elemen kedua dengan elemen kedua, dan seterusnya. Hasil pasangan tersebut kemudian diambil satu per satu oleh perulangan `for`, di mana variabel `lista` menyimpan nama warna dan `listb` menyimpan kode warnanya.

Di dalam perulangan, program mengisi dictionary hasil dengan format hasil[lista] = listb, sehingga nama warna menjadi key dan kode hexadecimal menjadi value. Terakhir, print(hasil) digunakan untuk menampilkan dictionary yang sudah terbentuk.

Output:

```
PS D:\Semester 2\Praktikum Algoritma dan Pemrograman\71251  
{'red': '#FF0000', 'green': '#008000', 'blue': '#0000FF'}
```

SOAL 3



```
1  fname = input("Masukkan nama file : ")  
2  
3  try:  
4      fhand = open(fname)  
5  except:  
6      print('file error tidak dapat dibuka:', fname)  
7      exit()  
8  
9  counts = {}  
10 for line in fhand:  
11     if line.startswith('From '):  
12         email = line.split()[1]  
13         counts[email] = counts.get(email, 0) + 1  
14  
15 print(f"{email}: {counts}")
```

Penjelasan:

Kode tersebut digunakan untuk membaca file dan menghitung berapa kali setiap email muncul di dalamnya. Pertama, program meminta pengguna memasukkan nama file, lalu mencoba membuka file tersebut menggunakan try-except. Jika file tidak ditemukan, program akan menampilkan pesan error dan berhenti.

Setelah file berhasil dibuka, program membuat dictionary kosong bernama counts untuk menyimpan jumlah kemunculan email. Program kemudian membaca file baris demi baris menggunakan perulangan for. Hanya baris yang diawali dengan "From " yang diproses.

Dari baris tersebut, alamat email diambil menggunakan `split()[1]`, karena email berada pada posisi kedua setelah teks dipisahkan berdasarkan spasi.

Selanjutnya, program menghitung jumlah kemunculan email dengan `counts.get(email, 0) + 1`. Jika email belum ada di dictionary, nilainya dimulai dari 0, lalu ditambah 1 setiap kali email ditemukan. Terakhir, program menampilkan hasil perhitungan email yang tersimpan di dalam dictionary.

Output:

```
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\71251214_raffiel\minggu12> python -u "d:\Semester_2\Praktikum Algoritma dan Pemrograman\71251214_raffiel\minggu12\soal3.py"
Masukkan nama file : mbox-short.txt
cwen@iupui.edu: {'stephen.marquard@uct.ac.za': 2, 'louis@media.berkeley.edu': 3, 'zqian@umich.edu': 4, 'rjlw@iupui.edu': 2, 'cwen@iupui.edu': 5, 'gsilver@umich.edu': 3, 'wagner
mr@iupui.edu': 1, 'antranig@caret.cam.ac.uk': 1, 'gopal.ramasammycook@gmail.com': 1, 'david.horwitz@uct.ac.za': 4, 'ray@media.berkeley.edu': 1}
```

SOAL 4

```
1 def hitung_email(nama_file):
2     email = {}
3     try:
4         with open(nama_file, 'r') as file:
5             for line in file:
6                 if line.startswith('From '):
7                     domain = line.split()[1].split('@')[1]
8                     email[domain] = email.get(domain, 0) + 1
9     except FileNotFoundError:
10        print('file error tidak dapat dibuka:', email)
11    return email
12
13 def main():
14     fname = input("Masukkan nama file : ")
15     email = hitung_email(fname)
16     print(email)
17
18 if __name__ == "__main__":
19     main()
```

Penjelasan:

Program diawali dengan fungsi `hitung_email(nama_file)` yang menerima nama file sebagai parameter. Di dalam fungsi tersebut dibuat dictionary kosong bernama `email` untuk menyimpan domain dan jumlah kemunculannya. Program kemudian menggunakan try-

except untuk menangani error saat membuka file. Pada bagian with open(nama_file, 'r') as file:, file dibuka dalam mode baca ('r') dan dibaca baris demi baris menggunakan perulangan for. Program hanya memproses baris yang diawali dengan "From " menggunakan startswith('From ').

Jika kondisi terpenuhi, alamat email diambil dengan line.split()[1], lalu dipisahkan lagi menggunakan split('@')[1] untuk mengambil bagian domain setelah tanda @. Contohnya, jika email adalah user@gmail.com, maka domain yang diambil adalah gmail.com. Setelah itu, program menghitung jumlah kemunculan domain menggunakan email.get(domain, 0) + 1. Jika domain belum ada di dictionary, nilainya dimulai dari 0, lalu ditambah 1. Jika file tidak ditemukan, bagian except FileNotFoundError akan menampilkan pesan error. Setelah proses selesai, fungsi mengembalikan dictionary email menggunakan return.

Pada fungsi main(), program meminta pengguna memasukkan nama file, lalu memanggil fungsi hitung_email() untuk memproses file tersebut. Hasil akhirnya ditampilkan dengan print(email).

Output:

```
PS D:\Semester_2\Praktikum Algoritma dan Pemrograman\71251214_raffiel\minggu12> python -u "d:\Semester_2\Praktikum\nerFile.py"
Masukkan nama file : mbox-short.txt
{'uct.ac.za': 6, 'media.berkeley.edu': 4, 'umich.edu': 7, 'iupui.edu': 8, 'caret.cam.ac.uk': 1, 'gmail.com': 1}
```